

* Bài giảng về Vibe Coding dịch Python code ra Matlab code,

Sưu tầm và Biên soạn bổ sung phần Vibe Coding

With the support of Gemini

Prof Happy AI

Date: May 9, 2026

Double-click (or enter) to edit

1. Tối ưu hóa hàm Boolean bằng thuật toán Quine-McCluskey Trong thiết kế mạch, việc rút gọn các biểu thức logic giúp giảm số lượng cổng (gate), từ đó tiết kiệm diện tích chip và năng lượng.

Bài toán: Nhập vào các số minterm, Python sẽ thực hiện rút gọn hàm để tìm biểu thức tối thiểu.

```
# Sử dụng thư viện Sympy để xử lý logic
from sympy import symbols, SOPform
from sympy.logic import simplify_logic

# Định nghĩa các biến logic
A, B, C, D = symbols('A B C D')

# Danh sách các minterm mà tại đó hàm F = 1
minterms = [0, 1, 2, 5, 8, 10, 13]

# Sử dụng SOPform (Sum of Products) để tìm biểu thức tối thiểu
simplified_expr = SOPform([A, B, C, D], minterms)

print(f"Hàm Boolean tối thiểu: {simplified_expr}")
```

Hàm Boolean tối thiểu: $(\sim B \ \& \ \sim D) \mid (B \ \& \ D \ \& \ \sim C) \mid (\sim A \ \& \ \sim B \ \& \ \sim C)$

* Để chuyển đổi mã Python sử dụng thư viện sympy sang MATLAB,

chúng ta sẽ sử dụng Logic Gates hoặc các hàm trong Image Processing Toolbox (như Quine-McCluskey) nếu có, nhưng cách phổ biến và trực quan nhất trong MATLAB là sử dụng hàm boolean hoặc giải quyết bằng bảng Karnaugh thông qua các script tối ưu hóa.

Tuy nhiên, MATLAB không có hàm SOPform trực tiếp giống hệt SymPy trong thư viện chuẩn. Cách tương đương nhất là sử dụng Symbolic Math Toolbox.

Dưới đây là đoạn mã MATLAB tương ứng:

Mã MATLAB (Sử dụng Symbolic Math Toolbox)

Double-click (or enter) to edit

```
% Định nghĩa các biến logic (Symbolic variables)
syms A B C D

% Trong MATLAB, việc tối thiểu hóa logic thường được thực hiện qua hàm simplify
% với thiết lập 'Steps' cao hoặc sử dụng các hàm hỗ trợ.
% Chúng ta sẽ thiết lập biểu thức logic từ các minterms.

% Danh sách minterms: 0, 1, 2, 5, 8, 10, 13
% Biểu diễn dưới dạng tổng các tích (SOP):
f = (~A & ~B & ~C & ~D) | (~A & ~B & ~C & D) | (~A & ~B & C & ~D) | ...
    (~A & B & ~C & D) | (A & ~B & ~C & ~D) | (A & ~B & C & ~D) | ...
    (A & B & ~C & D);

% Tối thiểu hóa biểu thức
simplified_expr = simplify(f, 'Steps', 10);

% Hiển thị kết quả
fprintf('Hàm Boolean tối thiểu:\n');
disp(simplified_expr);
```

2. Mô phỏng Bảng sự thật (Truth Table Generation) Để kiểm tra tính đúng đắn của một thiết kế mạch trước khi hiện thực hóa trên phần cứng (FPGA/ASIC).

Bài toán: Viết script Python để tự động tạo bảng sự thật cho một tổ hợp cổng logic phức tạp.

```
import pandas as pd
import itertools

def logic_circuit(A, B, C):
    # Giả sử mạch: Y = (A AND B) OR (NOT C)
    return (A and B) or (not C)

# Tạo tất cả các tổ hợp đầu vào (0, 1) cho 3 biến
inputs = list(itertools.product([0, 1], repeat=3))
results = []

for A, B, C in inputs:
    Y = int(logic_circuit(A, B, C))
    results.append([A, B, C, Y])

# Hiển thị bằng DataFrame cho trực quan
df = pd.DataFrame(results, columns=['A', 'B', 'C', 'Output (Y)'])
print(df.to_string(index=False))
```

A	B	C	Output (Y)
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

*

Việc chuyển đổi từ Python sang MATLAB khá thuận lợi vì MATLAB vốn được thiết kế để xử lý ma trận và bảng biểu rất mạnh mẽ.

Dưới đây là đoạn mã MATLAB tương đương, sử dụng tính năng Table để thay thế cho DataFrame của Pandas:

```
% 1. Tạo tất cả các tổ hợp đầu vào (Truth Table) cho 3 biến
% Sử dụng hàm 'ff2n' (Full Factorial) từ Statistics Toolbox
% hoặc tự tạo bảng logic đơn giản
[C, B, A] = ndgrid([0, 1], [0, 1], [0, 1]);
inputs = [A(:), B(:), C(:)];

% 2. Định nghĩa hàm logic (tương đương logic_circuit)
% Trong MATLAB: & là AND, | là OR, ~ là NOT
A_vals = inputs(:, 1);
B_vals = inputs(:, 2);
C_vals = inputs(:, 3);

Y = (A_vals & B_vals) | (~C_vals);

% 3. Chuyển đổi sang Table để hiển thị trực quan (giống DataFrame)
results = table(A_vals, B_vals, C_vals, double(Y), ...
    'VariableNames', {'A', 'B', 'C', 'Output_Y'});

% Hiển thị bảng
disp(results);
```

3. Thiết kế Bộ đếm (Counters) và Máy trạng thái hữu hạn (FSM) Máy trạng thái là linh hồn của các bộ vi xử lý. Python giúp chúng ta mô phỏng quá trình chuyển trạng thái.

Bài toán: Mô phỏng một bộ đếm nhị phân 3-bit hoặc một máy trạng thái điều khiển đèn giao thông.

```
class SimpleCounter:
    def __init__(self, bits):
        self.limit = 2**bits
        self.current_value = 0

    def tick(self):
        self.current_value = (self.current_value + 1) % self.limit
        return f"{self.current_value:03b}" # Trả về dạng nhị phân

counter = SimpleCounter(3)
print("Kết quả bộ đếm 3-bit:")
```

```
for _ in range(8):
    print(counter.tick())
```

```
Kết quả bộ đếm 3-bit:
001
010
011
100
101
110
111
000
```

Trong MATLAB, cấu trúc Class (Lập trình hướng đối tượng) có cú pháp hơi khác một chút so với Python (sử dụng từ khóa classdef). Dưới đây là đoạn mã MATLAB tương đương hoàn toàn với logic của bạn:

1. Tạo file Class: SimpleCounter.m Trong MATLAB, bạn cần lưu phần này vào một file riêng có tên là SimpleCounter.m.

```
classdef SimpleCounter < handle
    properties
        limit
        current_value = 0
    end

    methods
        % Hàm khởi tạo (__init__)
        function obj = SimpleCounter(bits)
            obj.limit = 2^bits;
            obj.current_value = 0;
        end

        % Hàm tick
        function bin_str = tick(obj)
            obj.current_value = mod(obj.current_value + 1, obj.limit);

            % Chuyển đổi sang nhị phân với 3 chữ số (tffơng đffơng :03b)
            bin_str = dec2bin(obj.current_value, 3);
        end
    end
end
```

2. File thực thi (Script): Bạn có thể chạy các dòng này ở Command Window hoặc lưu thành một file .m khác.

```
% Khởi tạo bộ đếm 3-bit
counter = SimpleCounter(3);

fprintf('Kết quả bộ đếm 3-bit:\n');

for i = 1:8
    disp(counter.tick());
end
```

Giải thích các điểm khác biệt: classdef ... < handle: Trong MATLAB, để thuộc tính của đối tượng thay đổi được bên trong phương thức (giống như self trong Python), class nên kế thừa từ class handle. Nếu không, mỗi lần gọi tick, giá trị current_value sẽ không được lưu lại cho lần tiếp theo.

mod(a, b): Tương đương với toán tử % trong Python để lấy số dư.

dec2bin(giá_trị, số_bit): Đây là hàm cực kỳ tiện lợi của MATLAB để chuyển số thập phân sang chuỗi nhị phân với độ dài cố định, thay thế cho cách format chuỗi f"{val:03b}".

Index: MATLAB sử dụng vòng lặp từ 1:8 thay vì range(8).

Mẹo nhỏ: Nếu bạn không muốn tạo file class phức tạp, bạn có thể sử dụng Persistent variables trong một function đơn giản, nhưng cách dùng classdef ở trên là chuẩn xác nhất để giữ đúng cấu trúc lập trình hướng đối tượng của mã gốc.

4. Phân tích độ trễ của mạch (Timing Analysis) Trong mạch số cao tốc, độ trễ truyền dẫn (propagation delay) qua các cổng là rất quan trọng để đảm bảo dữ liệu không bị lỗi.

Bài toán: Tính toán tổng độ trễ trên đường dẫn dài nhất (Critical Path).

```
# Định nghĩa độ trễ của các cổng (nanoseconds)
gate_delays = {'AND': 2.0, 'OR': 2.5, 'NOT': 1.0, 'XOR': 3.5}

# Cấu trúc mạch: Input -> NOT -> AND -> OR -> Output
path = ['NOT', 'AND', 'OR']
```

```
total_delay = sum(gate_delays[gate] for gate in path)
print(f"Tổng độ trễ trên đffờng dẫn: {total_delay} ns")
```

Tổng độ trễ trên đffờng dẫn: 5.5 ns

Trong MATLAB, cấu trúc dữ liệu tương đương với dictionary của Python là containers.Map. Tuy nhiên, đối với các bài toán đơn giản, bạn cũng có thể sử dụng kiểu dữ liệu struct.

Dưới đây là phiên bản sử dụng containers.Map để giữ đúng phong cách mã nguồn của bạn:

```
% 1. Định nghĩa độ trễ của các cổng (nanoseconds)
% Sử dụng containers.Map tffờng đffờng với Dictionary trong Python
gate_delays = containers.Map({'AND', 'OR', 'NOT', 'XOR'}, [2.0, 2.5, 1.0, 3.5]);

% 2. Cấu trúc mạch: Input -> NOT -> AND -> OR -> Output
path = {'NOT', 'AND', 'OR'};

% 3. Tính tổng độ trễ
total_delay = 0;
for i = 1:length(path)
    gate = path{i};
    total_delay = total_delay + gate_delays(gate);
end

% 4. Hiển thị kết quả
fprintf('Tổng độ trễ trên đffờng dẫn: %.1f ns\n', total_delay);
```

. Sinh mã Verilog tự động (Hardware Generation) Python có thể được dùng làm công cụ Meta-programming để tạo ra hàng nghìn dòng mã phần cứng một cách tự động (đặc biệt hữu ích trong các thiết kế lặp lại như bộ nhớ hoặc ma trận tính toán).

```
def generate_verilog_and_gate(num_inputs):
    inputs = ", ".join([f"in{i}" for i in range(num_inputs)])
    logic = " & ".join([f"in{i}" for i in range(num_inputs)])

    verilog_code = f"""
module and_gate_custom({inputs}, out);
    input {inputs};
    output out;

    assign out = {logic};
endmodule
"""
    return verilog_code

print(generate_verilog_and_gate(4))
```

```
module and_gate_custom(in0, in1, in2, in3, out);
    input in0, in1, in2, in3;
    output out;

    assign out = in0 & in1 & in2 & in3;
endmodule
```

* Trong MATLAB, chúng ta có thể sử dụng hàm strjoin để thay thế cho phương thức .join() của Python và sử dụng cú pháp nối chuỗi để tạo ra đoạn mã Verilog.

Dưới đây là đoạn mã MATLAB tương đương:

```
function verilog_code = generate_verilog_and_gate(num_inputs)
% Tạo danh sách các đầu vào (in0, in1, ...)
input_list = cell(1, num_inputs);
for i = 0:num_inputs-1
    input_list{i+1} = sprintf('in%d', i);
end

% Nối các thành phần bằng dấu phẩy và dấu &
inputs_str = strjoin(input_list, ', ');
logic_str = strjoin(input_list, ' & ');

% Tạo chuỗi Verilog (Sử dụng newline để ngắt dòng)
verilog_code = sprintf([...
    'module and_gate_custom(%s, out);\n' ...
    '    input %s;\n' ...
    '    output out;\n\n' ...
```

```
'        assign out = %s;\n' ...  
'endmodule'], inputs_str, inputs_str, logic_str);  
  
end  
  
% Gọi hàm và hiển thị kết quả (tương đương print)  
disp(generate_verilog_and_gate(4))
```

Giải thích các điểm khác biệt: cell và vòng lặp: MATLAB không có list comprehension trực tiếp như `[f'in{i}' for i in range(n)]`. Cách chính thống là tạo một cell array và dùng vòng lặp để điền giá trị.

`strjoin`: Đây là hàm cực kỳ hữu ích trong MATLAB để nối các phần tử trong một cell array thành một chuỗi duy nhất với ký tự ngăn cách (tương tự `",".join()` trong Python).

`sprintf`: MATLAB sử dụng `sprintf` để định dạng chuỗi với các trình giữ chỗ (placeholder) như `%s` (cho chuỗi) và `%d` (cho số nguyên), thay thế cho f-string của Python.

Nối dòng bằng `...`: Để viết một chuỗi dài trên nhiều dòng cho dễ đọc trong file `.m`, chúng ta sử dụng dấu ba chấm `...` để báo hiệu mã nguồn còn tiếp tục ở dòng dưới.

Double-click (or enter) to edit

Start coding or generate with AI.